

CampusGuard: COS 214 Practical 5

1. Team Members

- Hafani Ramagoma : u23605091
- Anke de Frey: u24611400
- Tshegofatso Mautjane: u25645006

2. System Description

2.1 Incident-Response Responsibilities

CampusGuard is the control-room system for a university campus.

It takes an emergency from the moment it is reported to the moment it is resolved, coordinating the people and the physical infrastructure that respond to it. The system carries five responsibilities.

1. **Registering and tracking incidents:** Every emergency becomes an Incident object owned by the IncidentCoordinator, carrying an identifier, a type such as a chemical spill or a lockdown, and a location.
An incident is not a passive record: it moves through a defined lifecycle of Reported, Active, Contained and Resolved, and its current stage determines which operations are legal at any moment.
A unit cannot be dispatched to an incident that has already been resolved, and an incident that no one has responded to cannot be marked contained.
2. **Dispatching response units:** Three kinds of units respond to incidents: SecurityTeam for threats and crowd control, MedicalTeam for casualties, and FacilityStaff for hazards and maintenance. Each unit tracks its own availability, so a unit already committed to one incident cannot be sent to another. The coordinator locates a free unit of the required type on request.
3. **Controlling physical access:** The campus is a hierarchy of buildings, floors and rooms. CampusGuard can lock, unlock or restrict any level of that hierarchy, and an instruction issued at building level takes effect in every room below it.
During a lockdown the affected building is sealed; when the incident is stood down the same area is released.
4. **Issuing alerts:** Warnings are broadcast to an affected area, with an urgency that reflects the severity of the incident. Alerts are raised when an emergency begins and cleared when it ends. Because the alert hardware is an existing campus installation that CampusGuard did not design, the system also has to work with equipment whose interface it does not control and whose failures it must report rather than ignore.
5. **Coordinating the response:** No responder decides on its own what the rest of the system should do. When a unit reports from the scene, that report is handed to a central coordinator, which decides what follows: a confirmed threat triggers a building lockdown, a shelter-in-place alert and a medical team on standby, while a contained hazard releases the area and stands every unit down. Individual units therefore never need to know about each other.

All five responsibilities are driven from a single operator console. Operator actions are recorded as they are issued, so the most recent action can be reversed if a report turns out to be a false alarm.

2.2 Demonstration Scenarios

main.cpp builds one campus and runs two emergencies through it, chosen so that between them every pattern in the system is exercised and every required failure case appears.

The world. The campus is built as a tree: two buildings, the IT Building and the Chemistry Building, each divided into floors, each floor holding individual rooms. Three response units are registered with the coordinator: SEC-01, MED-01 and FAC-01. The siren system is given a mapping from area names to the numeric zone codes the legacy hardware expects, including one zone deliberately configured as a faulty panel so that a hardware failure occurs at a predictable point in the run.

Scenario 1 — Chemical spill on Chemistry Floor 2. A hazard is reported and the operator triggers the evacuation workflow with a single call. That one call locates the affected floor in the campus tree, registers the incident, raises an evacuation alert through the siren system and dispatches a security team, all without the operator issuing each instruction separately. Security then reports arriving on scene, and the incident moves from Reported to Active. Facilities attend the hazard and report it contained, at which point the coordinator moves the incident to Contained and stands every unit down. The operator finally resolves the incident, completing the full lifecycle.

This scenario demonstrates the one-call workflow, the recursive area lookup, the translation of an alert into legacy hardware terms, and a complete pass through all four incident states.

Scenario 2 — Threat reported in the IT Building. The operator triggers the lockdown workflow, which seals the IT Building down to every individual room and broadcasts a lockdown alert. A security team is dispatched and the incident becomes Active. Security then confirms the threat, and this is the point where the coordination is visible: on that single report the coordinator locks down the building, issues a shelter-in-place alert and brings in a medical team, none of which the reporting unit asked for or knows about.

The scenario then produces the three required failure cases:

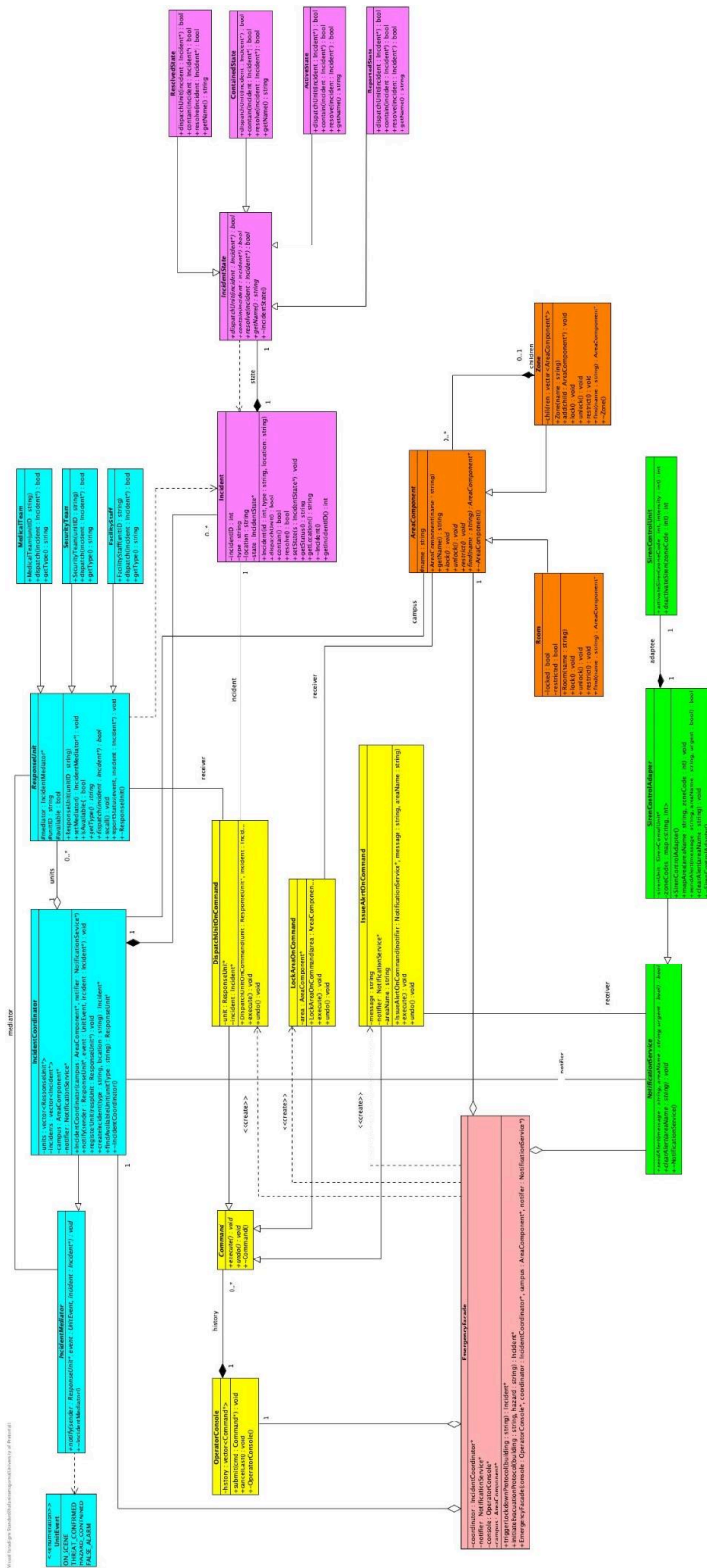
- **A hardware failure.** An alert is issued for the room whose siren panel is offline. The legacy unit returns an error code, which is translated into a reported failure rather than being silently discarded.
- **An unmapped area.** An alert is issued for an area with no siren zone configured at all. The request is refused before it ever reaches the hardware.

- **An invalid operation.** After the threat proves to be a false alarm and the incident is resolved, a further dispatch is attempted. The incident's current stage refuses it and the dispatch does not proceed.

Between the second and third of these, the operator cancels the most recent action, and that action is reversed.

Shutdown. Every object created in main is destroyed in an order that respects ownership, so that the run ends with no leaked memory. Section 8.2 gives the evidence.

Abstract Period: pre-transformational development of Pretest 11



4. GoF Participant Mapping

4.1 Command

GOF	CampusGuard classes
Command (abstract)	Command
ConcreteCommand	DispatchUnitOnCommand, LockAreaOnCommand, IssueAlertOnCommand
Invoker	OperatorConsole
Receiver	ResponseUnit subclasses, AreaComponent subclasses, NotificationService
Client	EmergencyFacade / main.cpp

4.2 Mediator

GOF	CampusGuard classes
Mediator (abstract)	IncidentMediator
ConcreteMediator	IncidentCoordinator
Colleague(abstract)	ResponseUnit
ConcreteColleague	SecurityTeam, MedicalTeam, FacilityStaff

4.3 Adapter

GOF	CampusGuard classes
Target	NotificationService
Adapter	SirenControlAdapter
Adaptee(legacy)	SirenContolUnit
Client	SecurityTeam, MedicalTeam, FacilityStaff

4.4 Facade

GOF	CampusGuard classes
Facade	EmergencyFacade
Subsystem classes	IncidentCoordinator, OperatorConsole, AreaComponent, NotificationService, Command hierarchy
Client	main.cpp

4.5 State

GOF	CampusGuard classes
Context	Incident
State (abstract)	IncidentState
ConcreteState	ReportedState, ActiveState, ContainedState, ResolvedState

4.6 Composite

GOF	CampusGuard classes
Component (abstract)	AreaComponent
Leaf	Room
Composite	Zone
Client	EmergencyFacade, IncidentCoordinator

5. Justification of Team-Selected Patterns

5.1 State

Design problem: An incident is not a static record. It progresses from being reported, through an active response, to being contained and finally resolved, and what may legally be done to it depends entirely on where it is in that progression. A unit may be dispatched to an incident that is still active, but not to one that has already been resolved; an incident may be marked contained only once a response is under way. Without a structure for this, Incident would hold a status field and every operation would begin by testing it.

Why State: The State pattern lets an object alter its behaviour when its internal state changes, so that the object appears to change its class. In CampusGuard, Incident is the Context and holds a pointer to an IncidentState. The four concrete states, ReportedState, ActiveState, ContainedState and ResolvedState, each implement dispatchUnit(), contain() and resolve(). Incident does not decide anything itself: incident->dispatchUnit() simply forwards to state->dispatchUnit(this), and the current state object decides whether the operation is legal, performs it, and where required installs the next state through setState(). Each state therefore owns its own rules, and the rules for one stage of the lifecycle live in one place.

Role in the workflow: State is the authority that the other patterns defer to. IncidentCoordinator (Mediator) never manipulates an incident's status directly; on receiving an event from a unit it calls contain() or resolve() and lets the state decide whether that transition is valid. A ResponseUnit dispatched through DispatchUnitOnCommand (Command) asks the incident before committing itself, so an attempt to dispatch to a resolved incident is refused at the state, not at the unit and not at the console. The state transitions are visible in the demonstration run, where incident #0 passes through all four stages and a later dispatch to the resolved incident #1 is rejected.

5.2 Composite

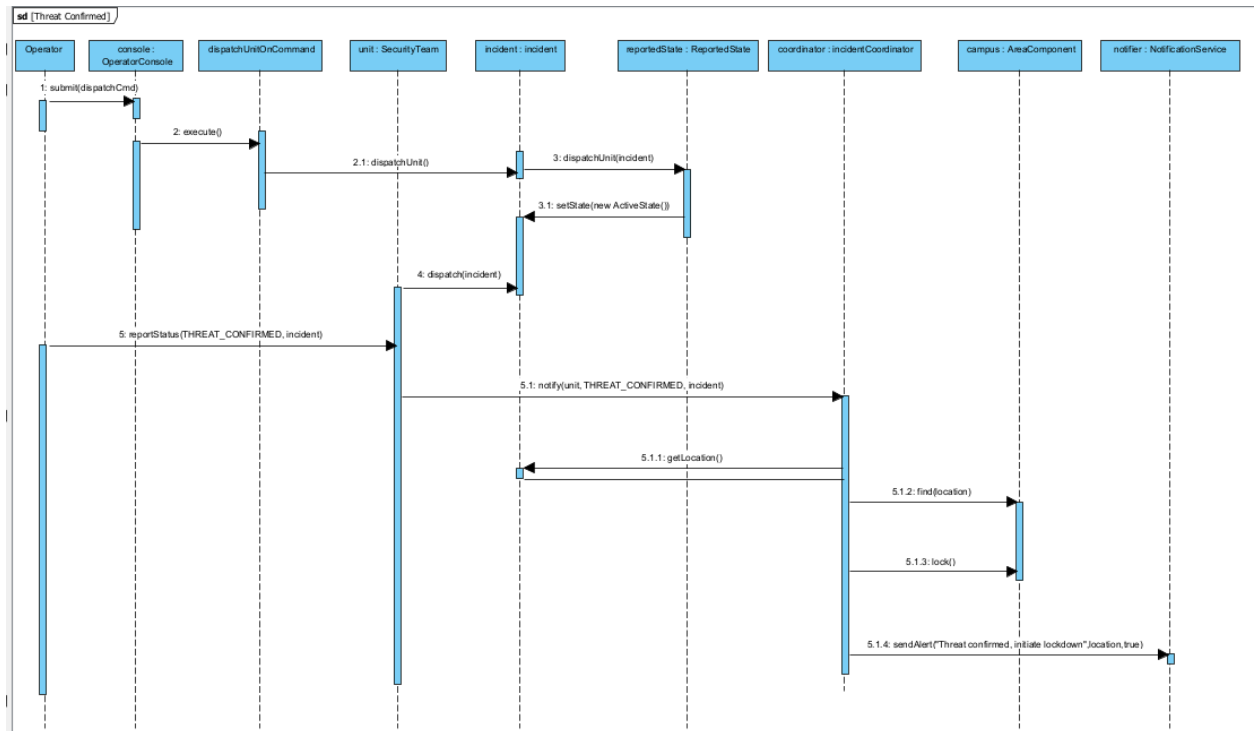
Design problem: CampusGuard needs to lock, unlock, or restrict different campus areas, such as rooms, floors, and buildings. The code performing these actions should work the same way regardless of the area's level.

Why Composite: The Composite pattern represents part-whole structures as a tree and allows individual objects and groups to be treated the same way. In CampusGuard, AreaComponent is the main interface, Zone represents groups such as buildings or floors, and Room is the individual area. For example, calling campus->find("IT Building")->lock() locks the building and everything inside it automatically through recursive lock() calls.

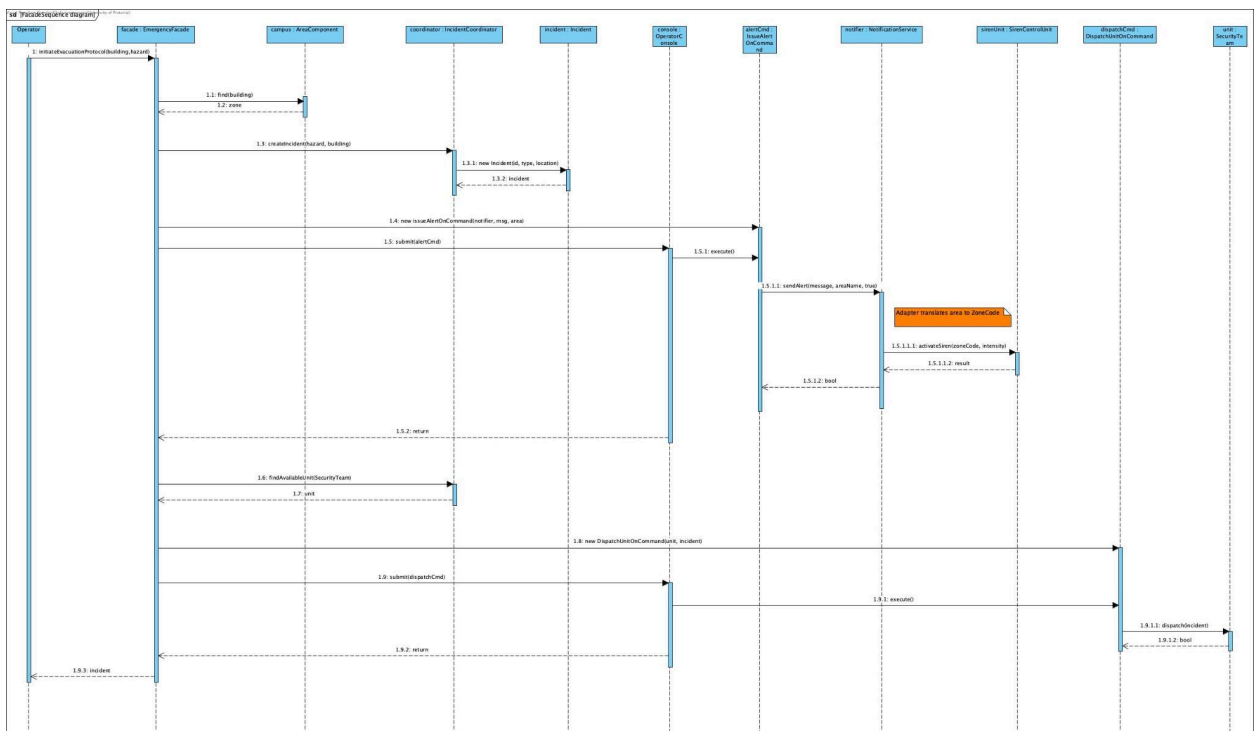
Role in the workflow: Composite works with other patterns in the system. LockAreaOnCommand uses AreaComponent to lock, unlock, and undo actions. IncidentCoordinator uses it to lock or unlock areas during incidents. EmergencyFacade uses it to restrict an area during evacuation. These classes only depend on AreaComponent, not on specific Zone or Room objects.

6. Behavioural Diagrams

6.1 Sequence Diagram: Command and Mediator Workflow

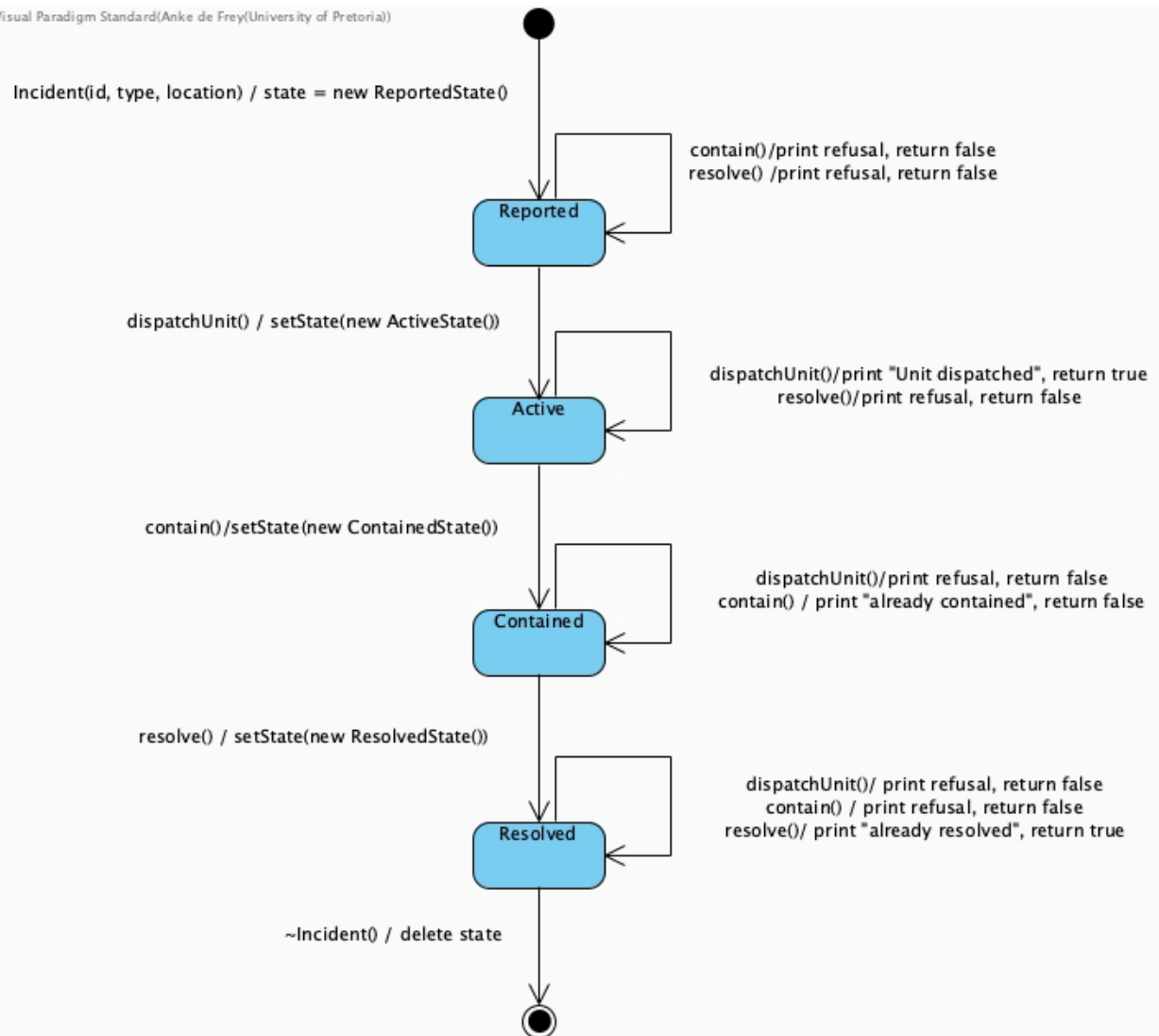


6.2 Sequence Diagram: Facade Workflow



6.3 State Diagram: Incident Lifecycle

Visual Paradigm Standard(Anke de Frey(University of Pretoria))



7. Design and Ownership Rationale

7.1 Pattern Interactions

The six patterns in CampusGuard do not operate as six isolated demonstrations — each one is a step in the same chain of collaboration, triggered by a single operator action. The clearest way to see this is to follow one call all the way through:

Facade → Composite. EmergencyFacade never manipulates the campus area tree directly; it calls `campus->find(building)` and receives back whichever `AreaComponent` (a `Room` or a whole `Zone`) matches, without needing to know how deep that area sits in the tree.

Facade → Mediator. The facade calls `coordinator->createIncident(...)` and `coordinator->findAvailableUnit(...)` rather than constructing an `Incident` or picking a `ResponseUnit` itself. The Mediator is the only thing that is allowed to decide which unit is available and to own the `Incident` once created.

Facade → Command → (Composite / Adapter / Mediator). This is where the patterns genuinely interlock rather than sitting side by side. The facade builds a `Command` and hands it to `OperatorConsole` (the invoker) without knowing what that command actually touches. But each concrete command's receiver is a participant in a different pattern: `LockAreaOnCommand`'s receiver is an `AreaComponent` (Composite), `IssueAlertOnCommand`'s receiver is a `NotificationService` (Adapter), and `DispatchUnitOnCommand`'s receiver is a `ResponseUnit` (Mediator colleague). One Command family is the single point of contact between the invoker and three otherwise-unrelated subsystems.

Command → Adapter. `IssueAlertOnCommand::execute()` calls `notifier->sendAlert(...)` purely through the `NotificationService` interface. It has no idea the real object underneath is a `SirenControlAdapter` translating that call into the legacy `SirenControlUnit::activateSiren(...)` call, with its own zone codes and numeric error codes. The Command pattern is what lets the Adapter be swapped in without the invoker or the facade changing at all.

Mediator → State. `IncidentCoordinator::notify()` never manipulates an incident's status field itself. It calls `incident->dispatchUnit()` / `contain()` / `resolve()`, and `Incident` simply forwards each call to its current `IncidentState`. The Mediator decides *that* something happened; the State pattern decides whether that transition is even legal from the incident's current status, and refuses it sensibly if not (e.g. dispatching to an already Resolved incident).

Colleague → Mediator (the reverse direction). A `ResponseUnit` never talks to another `ResponseUnit`, and never touches `Incident` directly either. `reportStatus()` only ever calls `mediator->notify(this, event, incident)` — the same one-directional discipline that keeps the Mediator, not the colleagues,

in charge of cross-unit coordination (e.g. escalating a THREAT_CONFIRMED report into dispatching a MedicalTeam backup).

The result: an operator only ever calls the Facade, but that single call ripples through Composite, Mediator, Command, Adapter and State before it's done — and a unit reporting back in the Mediator direction ripples through Mediator and State again. No pattern is reachable in isolation from main.cpp; each one is a link in the same chain.

7.2 Ownership and Destruction Policy

CampusGuard follows one consistent rule: a pointer is deleted only by whichever class *created* it (directly, or via a factory method). Anything received pre-built through a constructor or a registration call is treated as a reference and never freed by the holder.

- **IncidentCoordinator** owns its incidents (created via `createIncident()`) and deletes them in its destructor — but not its units, which arrive pre-built via `registerUnit()`.
- **OperatorConsole** owns every Command in history from the moment `submit()` is called; deleted individually on `cancelLast()`, or in bulk on destruction.
- **Zone** owns its children and deletes them recursively, cascading through the whole Composite tree in one loop.
- **SirenControlAdapter** owns and deletes the legacy `sirenUnit` it wraps — triggered polymorphically when `main.cpp` deletes it through the `NotificationService*` base pointer.
- **Incident** owns its state, deleted twice by design: once on every transition (`setState()`), and once for the final state in `~Incident()`.
- **EmergencyFacade** and **ResponseUnit** own nothing they hold (`console/coordinator/campus/notifier`, and `mediator`, respectively) — pure references to objects with independent lifetimes, hence no custom destructors needed.
- Every concrete **Command** treats its receiver (a unit, an area, the notifier) the same way: invokes behaviour on it, never owns or destroys it.
- **main.cpp** is the composition root, owning everything at the top level and deleting it all explicitly at shutdown, in an order safe against dangling references.

8. Debugging and Memory Evidence

8.1 GDB Investigation

```
Program received signal SIGSEGV, Segmentation fault.
0x000000000040184c in main () at test_area.cpp:15
15      campus->find("IT Building")->lock();
(gdb) bt
#0  0x000000000040184c in main () at test_area.cpp:15
(gdb) frame 1
No frame at level 1.
(gdb) print campus->find("IT Building")
Cannot resolve method Zone::find to any overloaded instance
The type: 'Zone *' isn't fully known to GDB. Please cast it directly to the desired typed in the function call.
```

The crash happens in main itself, on line 15. There's only one frame because the program never got into lock(). It crashed while trying to call it: calling a virtual method on a null pointer means reading that object's virtual function table, and there's no object. That's why frame 1 doesn't exist.

```
(gdb) break Zone::find
Breakpoint 1 at 0x402444: file Zone.cpp, line 42.
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /app/test_area
warning: File "/usr/local/lib64/libstdc++.so.6.0.32-gdb.py" auto-loading has been declined by your 'auto-load safe-path' set to "$debugdir:$datadir/auto-load".
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/aarch64-linux-gnu/libthread_db.so.1".

Breakpoint 1, Zone::find (this=0x4332b0, name=...) at Zone.cpp:42
42      vector<AreaComponent*>::iterator it;
(gdb) print this->name
$1 = {_M_dataplus = {<std::allocator<char>> = {<std::_new_allocator<char>> = {<No data fields>}, <No data fields>}, _M_p = 0x4332c8 "Campus"},
      _M_string_length = 6, {_M_local_buf = "Campus\000\000\000\000\000\000\000\000", _M_allocated_capacity = 126948234584387}}
(gdb) print this->children
$2 = {<std::Vector_base<AreaComponent*, std::allocator<AreaComponent*>> >> = {
      _M_impl = {<std::allocator<AreaComponent*>> = {<std::_new_allocator<AreaComponent*>> = {<No data fields>}, <No data fields>}, <std::Vector_base<AreaComponent
*, std::allocator<AreaComponent*>> >::Vector_impl_data = {_M_start = 0x433440, _M_finish = 0x433448,
      _M_end_of_storage = 0x433448}, <No data fields>}}, <No data fields>}
(gdb) finish
Run till exit from #0  Zone::find (this=0x4332b0, name=...) at Zone.cpp:42

Breakpoint 1, Zone::find (this=0x433300, name=...) at Zone.cpp:42
42      vector<AreaComponent*>::iterator it;
```

name is "Campus", as expected.

children holds exactly one pointer. _M_start is 0x433440 and _M_finish is 0x433448, and those are 8 bytes apart, which is one 64-bit pointer. That's the IT Building zone.

finish stopped at the breakpoint again, this time with this=0x433300. That's find being called on the child. So child loop does run, and the recursion works.

```
(gdb) list Zone::find
36      for (it = this->children.begin(); it != this->children.end(); ++it) {
37          (*it)->restrict();
38      }
39  }
40
41  AreaComponent* Zone::find(string name) {
42      vector<AreaComponent*>::iterator it;
43      for (it = this->children.begin(); it != this->children.end(); ++it) {
44          AreaComponent* result = (*it)->find(name);
45          if (result != nullptr) {
(gdb) print this->name
$3 = {_M_dataplus = {<std::allocator<char>> = {<std::_new_allocator<char>> = {<No data fields>}, <No data fields>}, _M_p = 0x433318 "IT Building"},
      _M_string_length = 11, {_M_local_buf = "IT Building\000\000\000\000", _M_allocated_capacity = 7236274653619049545}}
(gdb) next
43      for (it = this->children.begin(); it != this->children.end(); ++it) {
(gdb) next
44          AreaComponent* result = (*it)->find(name);
(gdb) next
45          if (result != nullptr) {
(gdb) next
43      for (it = this->children.begin(); it != this->children.end(); ++it) {
(gdb) next
44          AreaComponent* result = (*it)->find(name);
(gdb) next
Breakpoint 1, Zone::find (this=0x433350, name=...) at Zone.cpp:42
42      vector<AreaComponent*>::iterator it;
(gdb) █
```

```

41 AreaComponent* Zone::find(string name) {
42     // gdb 1 fix
43     if (this->name == name) {
44         return this;
45     }
46
47     vector<AreaComponent*>::iterator it;
48     for (it = this->children.begin(); it != this->children.end(); ++it) {
49         AreaComponent* result = (*it)->find(name);
50         if (result != nullptr) {
51             return result;
52         }
53     }
54     return nullptr;
55 }

```

Root cause: Zone::find() searched its children but never compared the zone's own name. When the child-search loop was added, it replaced the self-check if (this->name == name) return this; instead of being placed after it.

As a result, a search for "IT Building" reached the IT Building zone, found no matching child, and returned nullptr to the campus. main then called lock() on that null pointer, causing the segmentation fault at test_area.cpp:15.

Symptom: a segmentation fault at test_area.cpp:15.

Investigation: a breakpoint on Zone::find, print this->name showing "IT Building", and stepping with next.

Root cause: find searched the children but never compared the zone's own name.

Fix: the self-check, with its commit hash.

Verification: the clean rerun.

8.1.1 Incident::dispatchUnit() fires from two independent call sites.

-Symptom: Console output shows the "unit arrived" transition firing repeatedly with no clear single trigger - "Unit dispatched" prints multiple times per incident, with no indication of which unit or which incident caused it.

```

Incident reported
Unit dispatched
[Security] SEC-01 dispatched to Chemistry Floor 2
[Main] Incident status: Active
Unit dispatched
[Main] Incident status: Active

```

-Investigation:

```

(gdb) break Incident::dispatchUnit
Breakpoint 1 at 0xa1c0
(gdb) run
Starting program: /app/campusguard
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/aarch64-linux-gnu/libthread_db.so.1".
=== CampusGuard: Emergency Response Coordination ===

=====
SCENARIO 1: Chemical spill on Chemistry Floor 2
=====
[Adapter] "EVACUATE: Chemical spill reported at Chemistry Floor 2" for Chemistry Floor 2 -> activateSiren(22, 3)
[SirenUnit] ACTIVATE zone=22 intensity=3

Breakpoint 1, 0x000aaaaaaaa1c0 in Incident::dispatchUnit() ()
(gdb) bt
#0  0x000aaaaaaaa1c0 in Incident::dispatchUnit() ()
#1  0x000aaaaaaaa25c in SecurityTeam::dispatch(Incident*) ()
#2  0x000aaaaaaaa9b30 in DispatchUnitOnCommand::execute() ()
#3  0x000aaaaaaaa81c in OperatorConsole::submit(Command*) ()
#4  0x000aaaaaaaadd4 in EmergencyFacade::initiateEvacuationProtocol(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char>>, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char>>> ()
#5  0x000aaaaaaaa4038 in main ()

[(gdb) continue
Continuing.
Unit dispatched
[Security] SEC-01 dispatched to Chemistry Floor 2
[Main] Incident status: Active

Breakpoint 1, 0x000aaaaaaaa1c0 in Incident::dispatchUnit() ()
(gdb) bt
#0  0x000aaaaaaaa1c0 in Incident::dispatchUnit() ()
#1  0x000aaaaaaaaab010 in IncidentCoordinator::notify(ResponseUnit*, UnitEvent, Incident*) ()
#2  0x000aaaaaaaaad020 in ResponseUnit::reportStatus(UnitEvent, Incident*) ()
#3  0x000aaaaaaaa40b8 in main ()
(gdb) █

```

-Root cause: SecurityTeam::dispatch() calls incident->dispatchUnit() directly to decide whether the dispatch is allowed. But IncidentCoordinator::notify() also calls incident->dispatchUnit() when a unit reports ON_SCENE. So now an event is being told to the state machine twice, from two places that don't know about each other.

-Fix: Remove the direct call from SecurityTeam::dispatch() which should only mark the unit busy and the state transition should belong only to the Mediator reacting to a real ON_SCENE report.

```

//before
bool SecurityTeam::dispatch(Incident* incident) {
    if (!this->available) {
        cout << "[Security] " << this->unitID << " is not available" << endl;
        return false;
    }
    if (!incident->dispatchUnit()) { // the State decides
        cout << "[Security] Dispatch refused by incident state" << endl;
        return false;
    }
    this->available = false;
    cout << "[Security] " << this->unitID << " dispatched to " << incident->getLocation() << endl;
    return true;
}

//after
bool SecurityTeam::dispatch(Incident* incident) {
    if (!this->available) {
        cout << "[Security] " << this->unitID << " is not available" << endl;
        return false;
    }
    this->available = false;
    cout << "[Security] " << this->unitID << " dispatched to " << incident->getLocation() << endl;
    return true;
}

```

-Verification: The only remaining caller should now be the Mediator (No SecurityTeam::dispatch)
And no memory leak after valgrind check.

```

=====
SCENARIO 1: Chemical spill on Chemistry Floor 2
=====
[Adapter] "EVACUATE: Chemical spill reported at Chemistry Floor 2" for Chemistry Floor 2 -> activateSiren(22, 3)
[SirenUnit] ACTIVATE zone=22 intensity=3
[Security] SEC-01 dispatched to Chemistry Floor 2
[Main] Incident status: Reported

Breakpoint 1, 0x0000aaaaaa1c0 in Incident::dispatchUnit() ()
(gdb) bt
#0 0x0000aaaaaa1c0 in Incident::dispatchUnit() ()
#1 0x0000aaaaaa010 in IncidentCoordinator::notify(ResponseUnit*, UnitEvent, Incident*) ()
#2 0x0000aaaaaad020 in ResponseUnit::reportStatus(UnitEvent, Incident*) ()
#3 0x0000aaaaaa40b8 in main ()
(gdb) continue
Continuing.
Incident reported

Breakpoint 1, 0x0000aaaaaa1c0 in Incident::dispatchUnit() ()
(gdb) bt
#0 0x0000aaaaaa1c0 in Incident::dispatchUnit() ()
#1 0x0000aaaaaa560 in ReportedState::dispatchUnit(Incident*) ()
#2 0x0000aaaaaa1e0 in Incident::dispatchUnit() ()
#3 0x0000aaaaaa010 in IncidentCoordinator::notify(ResponseUnit*, UnitEvent, Incident*) ()
#4 0x0000aaaaaad020 in ResponseUnit::reportStatus(UnitEvent, Incident*) ()
#5 0x0000aaaaaa40b8 in main ()
(gdb) continue
Continuing.
Unit dispatched
[Main] Incident status: Active
Incident contained
[Main] Incident status: Resolved
Incident is resolved
[[Main] Incident status: Resolved

```

8.1.2 OperatorConsole::cancelLast()

-Symptom: cancelLast() reports "nothing to cancel" despite multiple commands submitted beforehand.

-Investigation: breakpoint at OperatorConsole.cpp:10, print history.size() -> 7, immediately followed by the false "nothing to cancel" message.

```
=====
FALSE ALARM: cancelling the last action
=====

Breakpoint 1, OperatorConsole::cancelLast (this=0xaaaaaad9b00) at OperatorConsole.cpp:10
10         if(!history.empty()){
(gdb) print history.size()
$1 = 7
(gdb) continue
Continuing.
[Console] Nothing to cancel - no actions in history
[Incident #1] Incident contained
[Incident #1] Incident is resolved
[Main] Incident status: Resolved

=====
INVALID OPERATION: dispatch to a resolved incident
=====
[Incident #1] Incident is resolved, no need to dispatch unit
[
=====
[ Shutting down
=====
[Inferior 1 (process 16) exited normally]
```

-Root cause: inverted condition - if(!history.empty()) should be if(history.empty()).

-Verification: history.size() should still show 7, but this time execution should proceed to actually pop and undo the last command instead of printing the false message.

IssueAlertOnCommand for "Sports Centre", calling its undo(), which calls notifier->clearAlert() through your Adapter, which correctly reports it can't clear an alert for a zone that was never mapped in the first place.

```
Before: history.size() = 7, but "[Console] Nothing to cancel..."
printed anyway
After: history.size() = 7, undo() actually runs -> "[Adapter] No
siren zone mapped..."
```

```

=====
FALSE ALARM: cancelling the last action
=====

Breakpoint 1, OperatorConsole::cancelLast (this=0xaaaaaaaaad9b00) at OperatorConsole.cpp:10
10         if(history.empty()){
(gdb) print history.size()
$1 = 7
(gdb) continue
Continuing.
[Adapter] No siren zone mapped for Sports Centre - alert NOT cleared
[Incident #1] Incident contained
[Incident #1] Incident is resolved
[Main] Incident status: Resolved

=====
INVALID OPERATION: dispatch to a resolved incident
=====
[Incident #1] Incident is resolved, no need to dispatch unit

=====
Shutting down
=====
[Inferior 1 (process 16) exited normally]

```

8.2 Valgrind evidence

```

==60==
==60== HEAP SUMMARY:
==60==    in use at exit: 0 bytes in 0 blocks
==60==   total heap usage: 118 allocs, 118 frees, 77,438 bytes allocated
==60==
==60== All heap blocks were freed -- no leaks are possible
==60==
==60== For lists of detected and suppressed errors, rerun with: -s
==60== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

9. Docker Setup

9.1 Dockerfile

```
Dockerfile > ...
1 FROM ubuntu:22.04
2
3 ENV DEBIAN_FRONTEND=noninteractive
4 RUN apt-get update && apt-get install -y --no-install-recommends \
5     g++ make valgrind gdb \
6     && rm -rf /var/lib/apt/lists/*
7
8 WORKDIR /app
9 COPY . .
10
11 RUN make clean && make
12
13 CMD ["/campusguard"]
```

9.2 docker-compose.yml

```
docker-compose.yml
  Run All Services
1 services:
  Run Service
2   campusguard:
3     build: .
4     image: campusguard:latest
5     container_name: campusguard
6     stdin_open: true
7     tty: true
8
9   #Run with: docker compose run --rm valgrind
  Run Service
10  valgrind:
11    image: campusguard:latest
12    profiles: ["debug"]
13    command: ["make", "valgrind"]
14
15  #Run with: docker compose run --rm gdb
  Run Service
16  gdb:
17    image: campusguard:latest
18    profiles: ["debug"]
19    stdin_open: true
20    tty: true
21    cap_add: ["SYS_PTRACE"]
22    security_opt: ["seccomp:unconfined"]
23    command: ["gdb", "./campusguard"]
```

9.3 Run Instructions

```
git clone https://github.com/Ankedefrey/COS214_Prac5.git
```

```
cd COS214_Prac5
```

```
docker compose up --build
```

```
docker compose run --rm valgrind
```

```
docker compose run --rm gdb
```

10. GitHub Collaboration

10.1 Repository Link

https://github.com/Ankedefrey/COS214_Prac5

10.2 Collaboration Reflection

We worked on a feature-branch model with no direct commits to main. The abstract headers and the Makefile were merged first through feature/core-interfaces, which froze the interfaces before anyone wrote an implementation. That turned out to be the decision that made parallel work possible: each member could code against the others' headers and compile long before those classes had any bodies. Branches were merged in dependency order, Composite and Adapter first, then Mediator, then Command and Facade, so that main built and ran after every merge.

Splitting the work by pattern rather than by file kept ownership clear and merge conflicts rare, with main.cpp reserved for a single owner at a time and handed over explicitly. What that split did not prevent were interaction defects: the patterns compiled independently but disagreed at their boundaries, and the three bugs documented in Section 8 all surfaced only once the branches were integrated. Each was diagnosed with GDB against a real failing run.

The most useful change we made was to our own design. The two additional patterns were originally sketched as Observer and Strategy, and we reconsidered them before implementation began, on the grounds that Observer overlapped with the Mediator we already had and that Strategy duplicated the Facade's workflows.

11. Contribution Statements

- Hafani: Contributed to the initial UML design for the system. Implemented the Mediator pattern (IncidentMediator, IncidentCoordinator) and the Facade pattern (EmergencyFacade), including the coordination logic in `notify()` that escalates confirmed threats and resolves false alarms through the incident state machine. Found and fixed three defects during final integration — a Makefile flag bug that had silently disabled debug builds project-wide, a Mediator/Command interaction bug causing duplicate state transitions, and an inverted condition in the undo logic — each verified with GDB and Valgrind evidence. Authored Sequence Diagram 2, covering the Facade's `initiateEvacuationProtocol()` workflow.
- Anke: Set up and maintained the GitHub repository, including the branching strategy, merge order and the team's git workflow guide, and authored the build and containerisation setup (Makefile, Dockerfile, `docker-compose.yml` and `README.md`).
Proposed the team's two additional patterns, State and Composite.
Contributed to finalisation of the UML class diagram.
Implemented the Composite pattern (AreaComponent, Zone, Room) and the Adapter pattern (NotificationService, SirenControlAdapter, SirenControlUnit), including the translation of area names and urgency levels into the legacy hardware's zone codes and intensities, and the handling of its integer error codes as reported failures.
Authored the initial `main.cpp`, building the campus hierarchy and siren configuration and scripting both demonstration scenarios. Carried out the GDB investigation of a segmentation fault caused by `Zone::find()` returning a null pointer, and fixed a defect in which a single `contain()` call cascaded through three state transitions, leaving the Contained state unreachable.
Produced the incident lifecycle state diagram and wrote the system description, the Composite and State justifications, and the Docker section of this report.
- Tshgofatso: Implemented the state and command design pattern, this includes the implementation and management of each individual concrete state and how they handle different incident functions along with the client, and the different concrete commands and its client. Created sequence diagram 1 which shows the flow of confirming and reporting a threat and how it is handled by the `dispatchUnit` Command, `securityTeam` and `incidentCoordinator` of the Mediator design pattern, and the `areaComponent`